

Guía de Programación Orientada a Objetos (POO)

Este documento detalla los pilares fundamentales de la POO, enfocándose en la estructura de las clases y su aplicación práctica mediante ejemplos específicos.

1. Definiciones Fundamentales

¿Qué es una Clase?

En el paradigma de la POO, una **clase** es una plantilla o "plano" que define la forma y el comportamiento de un objeto. No es el objeto en sí, sino el molde que determina qué datos almacenará y qué acciones podrá realizar.

Tipos de Clases

Existen diversas categorías de clases según su propósito en el software:

- **Clases Concretas:** Aquellas de las que se pueden crear instancias (objetos) directamente.
- **Clases Abstractas:** Clases que no pueden instanciarse y sirven como base para que otras clases hereden de ellas.
- **Clases de Utilidad:** Contienen métodos estáticos y no están diseñadas para crear objetos (ej. clases de cálculos matemáticos).
- **Interfaces:** (En algunos lenguajes) definen contratos de comportamiento que otras clases deben implementar.

Atributos de una Clase

Los **atributos** son las variables que pertenecen a una clase. Representan el **estado** o las características del objeto.

- *Ejemplo:* Color, tamaño, nombre, precio.

Métodos de una Clase

Los **métodos** son las funciones definidas dentro de una clase. Representan el **comportamiento** o las acciones que los objetos pueden ejecutar.

- *Ejemplo:* Caminar, calcular, guardar, mostrarInformacion.

2. Aplicación Práctica: Modelado de Clases

A continuación, se presentan los modelos para las entidades solicitadas, incluyendo sus atributos y métodos sugeridos.

A. Clase Alumno

Representa a un estudiante dentro de un sistema educativo.

- **Atributos:**
 - carnet: Identificador único (String).
 - nombre: Nombre completo (String).
 - fechaNacimiento: Para cálculos de edad (Date).
 - promedioAcumulado: Nota media (Float).
 - estadoMatricula: Activo, inactivo o egresado (String).
- **Métodos:**
 - matricularMateria(): Permite inscribir una nueva asignatura.
 - consultarCalificaciones(): Muestra el historial de notas.
 - calcularEdad(): Calcula los años del alumno basándose en su fecha de nacimiento.

B. Clase Docente

Representa al personal académico encargado de impartir clases.

- **Atributos:**
 - idEmpleado: Código de empleado (String).
 - especialidad: Área de conocimiento (String).
 - salario: Remuneración mensual (Decimal).
 - horario: Disponibilidad de horas (Map/Objeto).
- **Métodos:**
 - asignarCalificacion(): Permite ingresar notas a los alumnos.
 - registrarAsistencia(): Control de presencia en el aula.
 - subirMaterial(): Carga de recursos educativos al sistema.

C. Clase Materia (Asignatura)

Representa el contenido académico que se imparte.

- **Atributos:**
 - codigoMateria: Identificador de la asignatura (String).
 - nombreMateria: Título oficial (String).
 - unidadesValorativas: Créditos o peso de la materia (Int).
 - cuposDisponibles: Capacidad máxima de alumnos (Int).
- **Métodos:**
 - verificarRequisitos(): Revisa si el alumno tiene las materias previas aprobadas.
 - actualizarCupos(): Reduce o aumenta la disponibilidad tras una inscripción.
 - generarPrograma(): Muestra el contenido programático del ciclo.

3. Ejemplo de Relación

En un sistema real, estas clases interactúan: Un **Docente** imparte una **Materia**, y un **Alumno** se inscribe en esa **Materia**.